Angular Advanced
04 – Named router Outlets

Peter Kassenaar –
info@kassenaar.com

Named router outlets

- Multiple `<router-outlet>`'s in one component.
- Project different components in named outlets
  - `<router-outlet` **`name="list"`**`>…</router-outlet>`
  - `<router-outlet` **`name="detail"`**`>…</router-outlet>`
- State is reflected in URL
- Also know as *auxilliary routes*

OutletProject

localhost:4200/cities/(detail:1//list:cityList)

URL

Home Cities

List

- Groningen
- Hengelo
- Den Haag
- Enschede
- Heerlen

Details - Groningen

1

Name: Groningen

Province: Groningen

Highlights: Martinitoren

Outlets

../140-named-router-outlets

Preparation – set up routing module

- We use a simple setup:
  - One route pointing to the home route, `HomeComponent`
  - One route pointing to cities route, `CityComponent`
- CityComponent has two child routes
  - `CityListComponent`
  - `CityDetailComponent`

```
const routes: Routes = [
   {path: '', redirectTo: 'home', pathMatch: 'full'},
   {path: 'home', component: HomeComponent},
   {
      path    : 'cities', component: CityComponent,
      children: [
         {path: 'cityList', component: CityListComponent, outlet: 'list'},
         {path: ':id', component: CityDetailComponent, outlet: 'detail'}
      ]
   }
];
```

Set up router outlets

app.component.html – 1 unnamed <router-outlet>

```
<div class="container">
   <!--Main menu-->
   …
  <router-outlet></router-outlet>
</div>
```

city.component.html – 2 named <router-outlet>'s

```
<div class="row">
   <div class="col-md-6">
      <router-outlet name="list"></router-outlet>
   </div>
   <div class="col-md-4">
      <router-outlet name="detail"></router-outlet>
   </div>
</div>
```

Router navigate

Home button – static route (`app.component.html`)

```
<a class="btn btn-primary" routerLink="home">Home</a>
```

Cities button – dynamic route. Pass in the `outlets` object, and map the `list` outlet to `cityList` component and `detail` to `-1` for the moment.

```
<a class="btn btn-primary"
   [routerLink]="['/cities', {outlets: {'list': ['cityList'], 'detail': ['-1']}}]">
   Cities
</a>
```

Display CityList

- Default way of displaying data
  - Fetch cities via `cityService`.
  - We use the `async` pipe here

```
// city-list.component.ts
constructor(private router: Router,
         private cityService: CityService) {    }

ngOnInit(){
   this.cities=this.cityService.getCities();
}
```

```
<ul class="list-group">
   <li class="list-group-item"
      [class.selected]="city === currentCity"
      (click)="selectCity(city)"
      *ngFor="let city of cities | async">
      {{ city.name }}
   </li>
</ul>
```

Navigate to detail route

- Use `router` to show detail in the outlet:

```
selectCity(city: City) {
   console.log('navigate to: ', city.name);
   this.currentCity = city;
   this.router.navigate(['/cities', {outlets: {'detail': [city.id]}}])
}
```

Set up detail outlet using RouteParams

```
constructor(private route: ActivatedRoute,
         private  cityService: CityService) {
}

ngOnInit() {
   this.route.params.subscribe((params: { id: string }) => {
      this.id = +params.id;
      if (this.id === -1) {
         // Default: navigate to first city (or leave empty)
         this.id = 1;
      }
      this.city = this.cityService.getCity(this.id);
   })

}
```

Displaying the detail data

For your reference. This is easy:

```
<div *ngIf="city | async; else loading; let city">
   <ul class="list-group">
      <li class="list-group-item">Name: {{ city.name }}</li>
      <li class="list-group-item">Province: {{ city.province }}</li>
      <li class="list-group-item">Highlights: {{ city.highlights }}</li>
   </ul>
   <img src="assets/img/{{ city.name}}.jpg" alt="Foto van {{ city.name }}" />
</div>
<ng-template #loading>
   <div>
      <h3>Fetching city...</h3>
      <img src="assets/img/spinner.gif" alt="loading...">
   </div>
</ng-template>
```

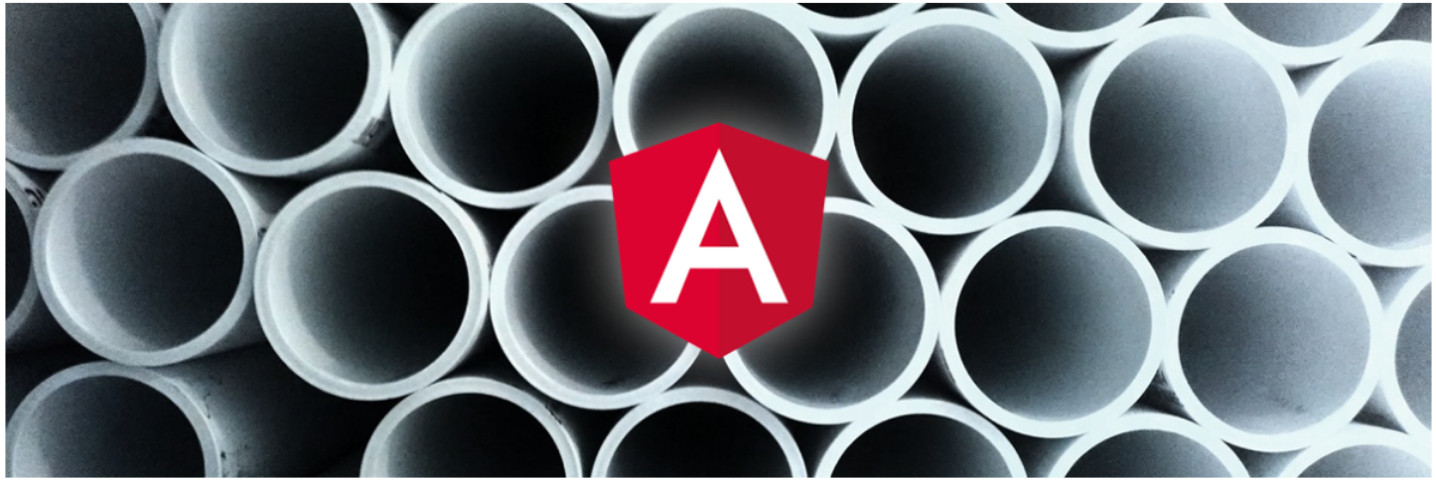
More info on named router outlets

ONEHUNGRYMIND

ABOUT

Dreams Do Come True! Named Router Outlets in Angular 2

Lukas Ruebbelke



Intro

I am generally a positive person that endeavors to say nice things about people and frameworks. In the broadest sense, I love Angular, and it has been an amazing tool to build some really cool things. Angular 2 has exceeded my expectations in a lot of ways.

http://onehungrymind.com/named-router-outlets-in-angular-2/

RANGLE.IO SERVICES PROCESS WORK EVENTS RESOURCES ABOUT BLOG INQUIRE

Type to search

master

Rangle.io : Angular 2 Training

Introduction

License

Why Angular?

The Architect's Guide to Angular

Creating Functional Forms

EcmaScript 6 and TypeScript Features

ES6

Classes

Refresher on 'this'

Arrow Functions

Template Strings

Inheritance

Delegation

Constants and Block Scoped V...

Using Auxiliary Routes

Angular supports the concept of auxiliary routes, which allow you to set up and navigate multiple independent routes in a single app. Each component has one primary route and zero or more auxiliary outlets. Auxiliary outlets must have unique name within a component.

To define the auxiliary route we must first add a named router outlet where contents for the auxiliary route are to be rendered.

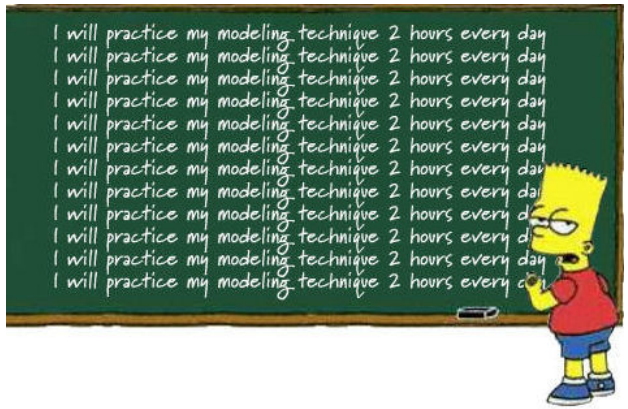
Here's an example:

```
import {Component} from '@angular/core';

@Component({
  selector: 'app',
  template: `
    <nav>
      <a [routerLink]="['/component-one']">Component One</a>
      <a [routerLink]="['/component-two']">Component Two</a>
      <a [routerLink]="[{ outlets: { 'sidebar': ['component-aux'] } }]">Component Aux</a>
    </nav>

    <div style="color: green; margin-top: 1rem;">Outlet:</div>
```

https://angular-2-training-book.rangle.io/handout/routing/aux-routes.html

Workshop

- Open `../140-named-router-outlets` and work from there
- OR: start a new project using routing and set it up using the previous slides.
- Add a new, named router outlet to `city.component.html`
- Create a new component to be shown in this outlet
- Create a link, navigating to this new outlet, showing the component
  - Place the link inside `city-list.component.html`.
  - Look at the `[routerLink]` notation!